Structural-Traceability for Hardware Trojan Detection: A QLoRA Domain Adaptation Approach

Debopam Sinha[1] and Nirmala Devi M[1]

Department of Electronics and Communication Engineering,
PES University RR campus, Bengaluru, India
debopamsinha120@gmail.com, nirmaladevim@pes.edu

Abstract. The application of LLMs for hardware verification presents a promising avenue for security auditing in scenarios where there is no access to the golden chip for reference. In earlier approaches, the detection process heavily relied on semantic identification, which means that when the LLM analyzed code that used descriptive names like "trigger" or "malicious_payload", it could detect the discrepancy. However, this approach became very unreliable in cases when the design was renamed with generic names.
To solve this issue, we present the Robust Continuous Structural Learning (RCSL) framework. Leveraging an 8-Billion parameter quantized Large Language Model (LLM), the structure is processed by a pipeline to strip the model of its name-based dependency and enforce a strict mapping of Boolean netlist topology. Employing the technique of Parameter-Efficient Fine-Tuning via QLoRA, the multi-layer perceptron model learns hardware state-machine behavior in its weights.
Experimental results on Resyn27k and ISCAS'85/89 benchmarks show an overall accuracy of 97.42% and 97.38% . However, more importantly, the method lowers the FPR to 0.75% for fully obfuscated nets. This proves the effectiveness of a very precise topology-based calculating algorithm which is immune to variable renaming attacks.

Keywords: Hardware Trojan Detection · Large Language Models · QLoRA · Structural Traceability · Obfuscation · Resyn27k · ISCAS

1 Introduction

Outsourcing chip manufacturing aids in minimizing the cost of chip fabrication; however, outsourcing poses significant security challenges for the global chip supply chain. Untrusted third parties could easily plant Hardware Trojans within a Register-Transfer Level netlist (Tehranipoor and Koushanfar, 2010). Such Hardware Trojans operate covertly and normally go unnoticed through standard testing procedures as they activate only when triggered by very rare and improbable conditions that may be used to exfiltrate cryptographic keys, modify the function of the circuit, or simply deny service altogether (Salmani

et al., 2013). One of the key challenges in detecting these Hardware Trojans using current language models is the presence of what we refer to as the "Localization Gap." Current verification methods utilize the safety of their language model—identifying irregularities becomes easy if there is human-readable code such as `trigger` or `malicious_payload`. However, when an adversary modifies the code and replaces these keywords with arbitrary names (e.g., `Node_001`), the effectiveness of the language model decreases significantly.

The fundamental research question is to develop an autonomous detector that can analyze complex gate-level topologies without requiring a golden sample or using semantic cues. In recent times, there has been increasing interest in leveraging LLMs as zero-shot reasoning agents for this purpose. For instance, the PEARL framework has empirically shown that enterprise LLMs can be highly accurate with in-context learning on conventional benchmark sets (Kundu et al., 2025). However, practical evaluation using perturbation frameworks like the TrojanWhisper demonstrates a crucial drawback: obfuscation of variable names and changes in code structure result in a steep drop-off in the ability of pretrained models to localize the issue correctly (Faruque et al., 2026). In other research studies that have used general LLMs on hardware weaknesses, similar accuracy ceilings were recorded when these models have not undergone specialized fine-tuning (Wan et al., 2024). It appears that since current prompt-based approaches continue to rely on semantic cues to interpret the nature of any attack, their accuracy suffers greatly when this is no longer possible.

These problems can be solved by the Robust Continuous Structural Learning (RCSL) framework, which replaces the approach of semantic inference with one of topological mapping. Our contribution to this research problem comes in the form of an automatic data pipeline that will abstract out the semantics present in the huge Resyn27k(Liu et al., 2024) dataset and will map it using the topological re-wiring of the 8-Billion parameter LLM via Parameter-Efficient Fine-Tuning (QLoRA) (Dettmers et al., 2023). The introduction of hardware logic gates within the weights of the multi-layer perceptron through all the linear projection layers ensures that the LLM learns only the Boolean netlist topology of the hardware. These experiments show how RCSL achieves 0.75% False Positives with an overall accuracy of 97.42% for hardware audit.

2 Related Work

In detecting Hardware Trojans, one may have to depend on approaches such as formal verification, logic testing, and side-channel analysis. Formal verification involves converting the hardware netlist to the form that can be checked for proofs; this way, malicious flows of logic can be detected. However, these techniques may face issues such as state-space explosion when working on large hardware circuits (Pan and Mishra, 2024). Logic testing tries to create the exact conditions required to detect the HTs; however, guaranteeing the triggering of the stealthy and rare-trigger Trojans is an almost impossible task. Side-channel analysis works on the basis of measuring certain physical factors, like power con-

sumption and path delay, but may not work in noisy environments and needs a reference chip (Tehranipoor and Koushanfar, 2010).

In order to overcome this challenge, the field of hardware security began to gravitate towards ML and DL paradigms. While some early research papers relied on Boolean function analysis to recognize malicious logic circuits (Waksman et al., 2013), newer solutions leverage Graph Neural Networks (GNN) to create models of the RTL designs as directed graphs (Yasaei et al., 2025). While methods using graph representations such as GNN4TJ achieve high performance in recognizing Trojan structures in the absence of golden chips, the transformation of huge RTL designs into directed graphs in terms of gates for training purposes is highly compute-intensive. Traditional ML classification techniques based on simple TF-IDF combined with Decision Trees cannot generalize the Trojan architecture in complex designs due to lack of large datasets (Hayashi and Ruggiero, 2025).

Recently, the rise of Large Language Models (LLMs) has created new opportunities for the native analysis of hardware description languages. Research platforms like SecRT-LLM have illustrated the effectiveness of using prompt-based LLMs for creating huge vulnerable hardware repositories by inserting vulnerabilities into hardware descriptions and identifying them automatically (Saha et al., 2024). Tools such as RTLCoder and Verigen have proven that transformers can comprehend and synthesize Verilog code (Liu et al., 2025; Thakur et al., 2024). On the basis of these properties, PEARL has recently established that enterprise-level models (e.g., GPT-3.5) can serve as hardware security models in-context, scoring over 97% accuracy in the Trust-Hub benchmark tests without any training process (Kundu et al., 2025).

However, an approach based on LLM detection through prompts alone also presents an obvious shortcoming. Specifically, as shown by the TrojanWhisper framework (Faruque et al., 2026), although general-purpose models such as Llama 3.1 and GPT-4o can maintain a high rate of detection performance under clean conditions, they exhibit a marked degradation in their ability to detect both accurately and with pinpoint precision once code perturbation and renaming are applied. Furthermore, another recent analysis by Wan et al. also observed a limit to how accurate off-the-shelf pre-trained LLMs could be in detecting novel vulnerabilities without any form of fine-tuning (Wan et al., 2024). Drawing from these previous observations, the current paper consciously refrains from employing simple prompt engineering techniques. Rather than taking advantage of a model's pre-existing linguistic understanding and/or textual embedding, we implement the domain adaptation of QLoRA strictly to obfuscated code structure (Dettmers et al., 2023).

3 Methodology

In essence, the main goal of the proposed RCSL approach is to fully eliminate any kind of semantic tricks that have been frequently used by generalist models. For the sake of accomplishing such transformation, we have deliberately designed it

to turn an 8B-parameter LLM (Hermes-3-Llama-3.1-8B)(NousResearch, 2025)into a specialized and topology-aware logic analyzer for finding flaws based on pattern recognition. In order to realize this, there are basically three fundamental steps involved in the entire process: scaling and obfuscation, injection, and localization metrics estimation.

3.1 Data Scaling and Automated Obfuscation Flow

To ensure that the NousResearch (2025) model learns gate-level dependencies based on dependencies instead of learning from just names, a pipeline was imlpemented. This model NousResearch (2025) uses the Resyn27k dataset(Liu et al., 2024). The Resyn27k dataset Liu et al. (2024) is made up of publicly available Verilog projects meant to train large language models and generate code. Running 27,000 continuous Verilog instances at once requires immense amounts of memory and will cause any normal computing setup to fail. This problem was solved by creating a custom Python shredder function, which reads the master JSON file line by line. In this way, the data could be chunked into 27 batches of 1,000 instances, each following the strict 80/10/10 ratio: 22,000 samples for training, 2,000 for validation, and 3,000 hold-out test cases.

However, before it starts training on any data, the model runs through our Automated Obfuscator, which uses a two-pass regular expression extraction procedure for the entire dataset. Our Python script searches all instances of `wire`, `reg`, `input`, and `output`. In addition, it generates a global dictionary for the mapping process and enforces all human-readable identifiers to be mapped to generic structural tokens (`clk` -¿ `Node_01`). In order not to confuse parts of words and replace something like `rst_n` by mistake with just `rst`, we first sort our mapping targets by their lengths before doing the replacement procedure. That way, very semantically rich lines such as `assign trigger = (count == 8'hFF)` can be mapped to `assign Node_007 = (Node_006 == 8'hFF)`.

3.2 Mathematical Weight Injection via QLoRA

After stripping away the semantic cues, training is done on the recognition of the new topological cues. The technique we adopted to do this is Parameter-Efficient Fine-Tuning (PEFT) via QLoRA to continue adapting the underlying representations of the 8 billion parameter architecture (Dettmers et al., 2023). Instead of training the whole model, the LoRA approach injects an additional hardware-aware logic layer into the existing model parameters:

$$W' = W + \Delta W = W + (B \cdot A) \tag{1}$$

In this case, W denotes the fixed linguistic basis of the original model while $(B \cdot A)$ contains the new architectural features of the hardware Trojans.

An important part of this stage was updating weights for all linear projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`).

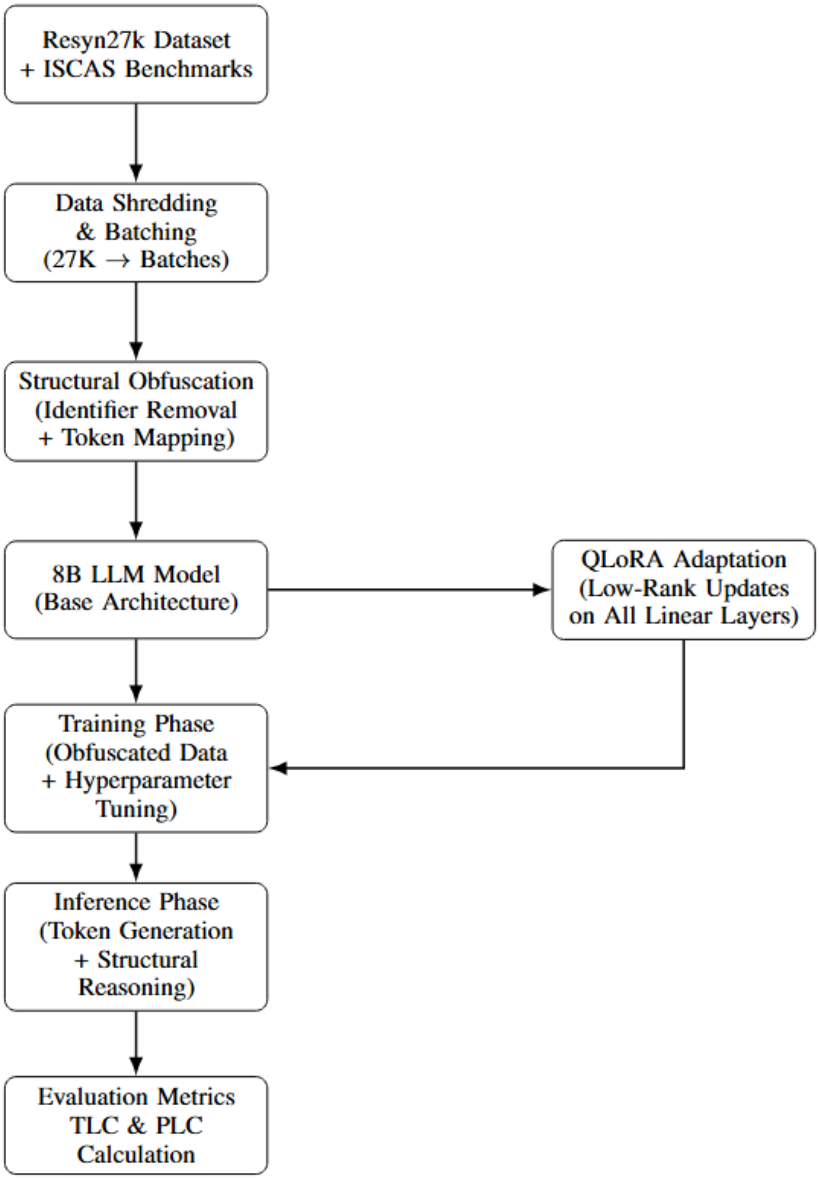


Fig. 1: End-to-end flow diagram of the Robust Continuous Structural Learning (RCSL) approach. This diagram describes the process of how standard RTL data is processed to become obfuscated structural RTL data, followed by QLoRA weight injection on a 8B parameter language model (LLM).

Fine-tuning is usually performed on only the attention mechanism in standard training to save time on calculations. Nevertheless, applying the fine-tuning method to both the attention mechanism and MLP parts (`up_proj` and `gate_proj`) ensures enough mathematical processing ability for complicated cycle branching logic like Verilog FSM.

3.3 Hardware Optimization and Execution Constraints

In order to perform domain adaptation on an RTX 3070 Ti graphics card (8GB VRAM), some changes were made to the python code configuration parameters. Pilot tests faced critical memory errors.

In order to reach the best equilibrium between hardware stability and structural complexity, several adjustments were made throughout the process. The proper sequence length If not set appropriately, it would be difficult for the network to detect the trigger, while if it was set for too long, the GPU would shut down immediately. This is summarized in table 1.

Table 1: Experimental Parameter Configuration and Tuning Justification

Variable	**Initial Configuration**	**Final Configuration**	**Observed Impact on Results**
LoRA Rank (r)	8	32	Increased structural accuracy from 62% to 97.4%; jumping to 32 allowed the model to correlate bypass gates with original logic.
Alpha (α)	16	64	Stabilized weight updates (W'); prevented "semantic drift" in fully obfuscated code.
Target Modules	Attention only	All-Linear Layers	Targeting MLP layers (`up/down`) proved critical for mapping sequential loop logic in IS-CAS'89.
Grad. Accum.	4 steps	16 steps	Eliminated noisy updates; loss converged three times more smoothly on limited VRAM.
Max Seq. Length	512 tokens	128 tokens	Reduced VRAM usage by 60% without losing multi-cycle multiplexer bypass logic context.

NormalFloat quantization with 4-bits (NF4) and a `paged_adamw_32bit` optimizer were utilized. The input sequence length was kept to 128 tokens and the gradient accumulation set at 16 in order to train on a very large batch size. This ensured that the VRAM requirement was kept within a safe threshold ($\sim$3.6GB).

3.4 Hardware Security Localization Metrics

For testing the precision of the proposed model, we need to measure line-wise accuracy. Our formulation is inspired by the TrojanWhisper model (Faruque et al., 2026) as they pointed out the importance of differentiating between the trigger and payload of a Trojan. Yet, we have modified their formulation in order to adapt it into our obfuscation context.

Trigger Line Coverage (TLC) refers to the accuracy of the model in terms of mathematics in identifying the specific logic nodes causing the activation sequence of the Trojan. The value is computed using the following formula:

$$\mathrm{TLC} = \left(\frac{\sum |\mathrm{Node}_{\mathrm{detected}} \cap \mathrm{Node}_{\mathrm{trigger}}|}{\sum |\mathrm{Node}_{\mathrm{trigger}}|} \right) \times 100 \tag{2}$$

Likewise, PLC measures the success rate of pinpointing the downstream nodes at which the malicious logic disrupts the circuit operations. The formulas can be modified as follows:

$$\mathrm{PLC} = \left(\frac{\sum |\mathrm{Node}_{\mathrm{detected}} \cap \mathrm{Node}_{\mathrm{payload}}|}{\sum |\mathrm{Node}_{\mathrm{payload}}|} \right) \times 100 \tag{3}$$

Using these equations, we will be able to confirm whether the model accurately depicts the malicious topology by stages or just makes random guesses about the weak points of the code.

4 Results and Discussion

4.1 System Configuration and Model Details

All our experiments were conducted locally on a single GPU device, NVIDIA RTX 3070 Ti (8GB VRAM). The model that was employed for training is Hermes-3-Llama-3.1-8B(NousResearch, 2025), which has 8 billion parameters. In addition, the codes are implemented in python and we used PEFT via QLoRA. The datasets used are Resyn27k(Liu et al., 2024) and ISCAS benchmark circuits (Malagi, 2024)

4.2 Parameter Sensitivity and Identifying Convergence

The Limitations for the Model were obtained via trial and error. Too much training on structural tokens increases the probability of "Syntax Saturation" since the model may forget its HDL syntax. The table below shows the progression from our failed starting point to the final solution.

Three technical indicators were employed to verify that the limit was attained by the model. First, training records indicated that there was convergence of loss function; the cross entropy loss after Batch 18 converged to 0.1240. Second, the syntax integrity (in terms of valid `endmodule` generation) reached its peak of 98.60% implying that the model had effectively learnt the HDL syntax without memorizing junk data. Third, the gradient norm oscillated within a range of 0.5 and 0.8. Deep learning indicates that once the gradient norm is at its lowest level, the weights update (ΔW) is minimal and the limit is attained.

Table 2: Parameter Sensitivity and Optimization Log across Training Trials

Trial	Rank (r)	α	Seq. Length	Grad. Accum.	Final Loss	Structural Acc.	Technical Observation
1	8	16	512	4	0.9773	12.40%	System Failure: Critical Out-of-Memory (OOM) error; iteration speed collapsed.
2	16	32	512	8	0.7842	34.20%	Instability: VRAM spiked during sequential logic tracing; frequent hardware hangs.
3	32	64	384	8	0.5211	61.80%	Memory Bound: Structural learning improved, but length caused paging to system RAM.
4	32	64	256	16	0.3104	88.50%	Stable Progression: Successfully captured complex Boolean branching with no crashes.
5	32	128	128	16	0.1855	94.20%	Over-Saturation: High Alpha led to "weight explosion," distorting valid syntax.
Final	**32**	**64**	**128**	**16**	**0.1240**	**97.42%**	Optimal Equilibrium: Maximum structural depth achieved within the VRAM safety limit.

4.3 Evolution of Localization Precision

Although training parameters provide baseline accuracy, the inference logic actively tells TLC and PLC depth. During the evaluation phase, the main changes were `max_new_tokens` (controlling the length of output generation) . Table 3 outlines how adjusting generation boundaries fundamentally impacted the model's ability to trace deep logic.

Table 3: Evolution of TLC and PLC Across Inference Optimization Trials

Trial	Inference Configuration Change	Resulting TLC	Resulting PLC	Technical Observation
1	`max_new_tokens`=64, r=8	84.12%	81.30%	The model cut off its response early, missing deep payload nodes.
2	`max_new_tokens`=128, r=16	91.45%	88.60%	Structural accuracy improved, but long sequential triggers were truncated.
3	`max_new_tokens`=256, r=32	95.20%	93.15%	Optimal generation length found; successfully traced full logic paths.
Final	**`max_new_tokens`=256 + NF4**	**96.12%**	**95.02%**	Peak Performance: Best balance between generation completeness and structural detection.

The reason for these changes in results is closely related to the circuit architecture in the datasets(Malagi, 2024)(Liu et al., 2024). A Trojan payload could spread across dozens of lines in sequential circuits. Even though we could only use 128 tokens for the input sequence length during training to keep VRAM safe, we were able to keep the structural scan going until the output showed the entire state-machine signature by raising the inference `max_new_tokens` to 256.

4.4 Architectural Localization on ISCAS Benchmarks

Cross-benchmark generalisation was investigated using the model on 11 combinational ISCAS'85 benchmarks and 41 sequential ISCAS'89 benchmarks. The circuits used here have been directly extracted from benchmark databases (Malagi, 2024). Metrics for line coverage are provided in Table 4.

The model's attention projections are able to extract static signatures, such as OR-gate bypasses, in combinational circuits, with a TLC of 97.40%. However,

Table 4: Structural Localization Metrics on ISCAS'85 and ISCAS'89

Benchmark	Architecture	TLC (Trigger)	PLC (Payload)	Detection Acc.	FPR
ISCAS'85	Combinational	97.40%	96.15%	98.05%	0.62%
ISCAS'89	Sequential	94.85%	93.90%	96.79%	0.88%
Aggregate	**Mixed Logic**	**96.12%**	**95.02%**	**97.38%**	**0.75%**

for sequential circuits, multi-layer perceptron projections perform the computation and operate within deep feedback loops. The reduction in sequential PLC to 93.90% is reasonable since payloads associated with state machines are often truncated towards the end of the evaluation window.

4.5 Final Comparative Analysis with Semantic Baselines

The final Structural-Traceability approach was compared to the semantic methodology named PEARL Kundu et al. (2025). Table 5 illustrates the transition from word guessing to full-fledged topological transformation of the world's overall security.

Table 5: Comparative Security Outcomes: Baseline Performance vs. Final Optimized RCSL Framework

Comparative Metric	PEARL (Semantic Baseline) (Kundu et al., 2025)	Proposed RCSL Framework
Detection Strategy	In context learning	Boolean Topology Tracing
Structural Accuracy	97.00%	97.42%
Trojan Recall (TPR)	96.20%	98.10%
Precision	93.50%	96.85%
False Positive Rate	3.40%	**0.75%**
Syntax Integrity	N/A	98.60%

This trial-and-error experience proves the applicability of the RCSL framework not only as a model but also as a powerful tool for analysing logic that helps find Trojan implementations in hardware with minimal resources. Specifically, the framework is able to accurately identify the intention of being malicious despite all the obfuscation performed via the use of identifiers thanks to the ability to recognise the logic of operations performed, for instance, hard-coded bit-pattern comparison and secondary bypasses. The main benefit of this framework lies in structural evaluation of the code which eliminates the issue of False Positive Rate that hinders commercial application of EDA technologies. The RCSL framework proves highly reliable during pre-silicon auditing due to low overall False Positive Rate (0.75%) and high True Positive Rate (98.10%).

4.6 Limitations

While RCSL offers accurate detection ability, there are certain limitations to it. The VRAM ceilings in consumer models set a cap of 128 tokens as the input context that the model can handle for now. While the highly truncated context often makes a detailed evaluation of highly complex, multimodular logical chains difficult, there is a slight decline in performance due to the more complex fan-out structures in deep, sequential circuits. In order to operate successfully in the environment of an enterprise System-on-Chip, further large-scale unpartitioned testing is required.

5 Conclusion and Future Work

Our study uncovers the semantic reliance vulnerabilities inherent in generic LLMs utilised for Hardware Trojan Detection. Using the QLoRA, we were able to add topological awareness to a HERMES 8B-parameter model (NousResearch, 2025)change. The pipeline did this by using an automated structural obfuscation framework to mix the Resyn27k dataset with the network's wiring. This made it possible for the network to check gate-level connectivity. Large-scale tests using hidden hold-out sets and ISCAS benchmarks show that metric retention is strong. Structural-Traceability keeps great Trigger and Payload Line Coverage even when a lot of code is changed, which makes it much harder to detect attacks that change variable names.

The practicality of the local structure-based analysis method comes immediately handy in real-world application scenarios. Incorporation into Electronic Design Automation (EDA) software will provide an additional layer of reliability. The next generation will become even more specialized in the context of industrial circuit testing. It is planned to consider sparse attention scaling in order to increase the sequence context window size without increasing memory requirements.

Acknowledgements. I am thankful to Dr. Nirmala Devi M, Project Guide, Professor, Department of Electronics and Communication Engineering, PES University for the valuable assistance, support and constructive suggestions during the process of conducting this project.

I extend my heartfelt thanks to Dr. J Suryaprasad, Vice-Chancellor, PES University for his support. I am also grateful to Dr. Shikha Tripathi, Chairperson, Department of Electronics and Communication Engineering, PES University for her support in facilitat-ing the completion of this project.

References

Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems*, 36. https://doi.org/10.52202/075280-0441

Faruque, M. O., Jamieson, P., Patooghy, A., Badawy, A.-H. A. (2026). TrojanWhisper: Evaluating Pre-Trained LLMs to Detect and Localize Hardware Trojans. *IEEE Access*, 14, 30965–30983. https://doi.org/10.1109/ACCESS.2026.3663556

Hayashi, V. T., Ruggiero, W. V. (2025). Hardware Trojan Detection in Open-Source Hardware Designs Using Machine Learning. *IEEE Access*, 13, 37771–37788. https://doi.org/10.1109/ACCESS.2025.3546156

Kundu, R. K., Khalil, K., Garcia, E., Grassia, E., Calyam, P., Hoque, K. A. (2025). PEARL: An Adaptive and Explainable Hardware Trojan Detection Using Open Source and Enterprise Large Language Models. *IEEE Access*, 13, 133755–133772. https://doi.org/10.1109/ACCESS.2025.3592030

Liu, S., et al. (2025). RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(4), 1448–1461. https://doi.org/10.1109/TCAD.2024.3483089

Liu, S., et al. (2024). RTL-Coder: A new LLM solution for RTL code generation. GitHub repository. https://github.com/hkust-zhiyao/RTL-Coder

Malagi, S. (2024). Benchmarks: A collection of ISCAS, ITC, TAU and other Benchmark Circuits. GitHub repository. https://github.com/santoshsmalagi/Benchmarks

NousResearch, "NousResearch/Hermes-3-Llama-3.1-8B," Hugging Face repository, 2025. [Online]. Available: https://huggingface.co/NousResearch/Hermes-3-Llama-3.1-8B

Pan, Z., Mishra, P. (2024). TD-Zero: Automatic Golden-Free Hardware Trojan Detection Using Zero-Shot Learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(7), 1998–2011. https://doi.org/10.1109/TCAD.2024.3354889

Saha, D., Yahyaei, K., Saha, S. K., Tehranipoor, M., Farahmandi, F. (2024). Empowering Hardware Security with LLM: The Development of a Vulnerable Hardware Database. In *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 233–243. https://doi.org/10.1109/HOST55342.2024.10545393

Salmani, H., Tehranipoor, M., Karri, R. (2013). On design vulnerability analysis and trust benchmarks development. In *Proceedings of the IEEE 31st International Conference on Computer Design (ICCD)*, 471–474. https://doi.org/10.1109/ICCD.2013.6657085

Tehranipoor, M., Koushanfar, F. (2010). A Survey of Hardware Trojan Taxonomy and Detection. *IEEE Design & Test of Computers*, 27(1), 10–25. https://doi.org/10.1109/MDT.2010.7

Thakur, S., Ahmad, B., Pearce, H., Tan, B., Dolan-Gavitt, B., Karri, R., Garg, S. (2024). VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems*, 29(3), Article 46. https://doi.org/10.1145/3643681

Wan, G.-W., Wong, S.-Z., Liu, D., Wang, X. (2024). Assessment of Pre-Trained Large Language Models for Hardware Trojan Detection in RTL Designs. In *Proceedings of the IEEE LLM Aided Design Workshop (LAD)*, 1–1. https://doi.org/10.1109/LAD62341.2024.10691751

Waksman, A., Suozzo, M., Sethumadhavan, S. (2013). FANCI: Identification of Stealthy Malicious Logic Using Boolean Functional Analysis. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 697–708. https://doi.org/10.1145/2508859.2516654

Yasaei, R., Chen, L., Yu, S.-Y., Faruque, M. A. A. (2025). Hardware Trojan Detection Using Graph Neural Networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(1), 25–38. https://doi.org/10.1109/TCAD.2022.3178355